Unity Code Submission

This document lists all C# scripts used in the two exercises.

Exercise 1: Nature Scene (Transforms and Camera)

Scripts extracted from Assets/Scripts in the Nature scene project.

EnvironmentSpawner.cs

Exercise: Exercise 1: Nature Scene

Class: EnvironmentSpawner

Purpose: Procedurally spawns environment prefabs such as rocks, trees, and flowers inside a bounded area, with rules to keep a playable corridor clear and optional reproducible seeding.

Code:

```
// ===== START OF FILE: EnvironmentSpawner.cs =====

using UnityEngine;

public class EnvironmentSpawner : MonoBehaviour
{
    [Header("Prefabs")]
    public GameObject rockPrefab;
    public GameObject treePrefab;
    public GameObject flowerPrefab;

    [Header("Parents (optional but recommended)")]
    public Transform rocksParent;
    public Transform treesParent;
    public Transform flowersParent;

    [Header("Area Settings")]
    public float areaHalfSize = 45f;       // square area: -half..+half on X and Z
    public float trailHalfWidth = 6f;      // keep the center corridor clear (along Z)

    [Header("Counts")]
    public int rockCount = 40;
    public int treeCount = 20;
    public int flowerCount = 8;

    [Header("Scaling")]
    public Vector2 rockScaleRange = new Vector2(0.6f, 2.0f);
    public Vector2 treeScaleRange = new Vector2(0.9f, 1.4f);

    [Header("Reproducibility (optional)")]
    public bool useSeed = true;
```

```
public int seed = 12345;

void Start()
{
    if (useSeed) Random.InitState(seed);

    SpawnRocks();
    SpawnTrees();
    SpawnFlowersOnTrail();
}

void SpawnRocks()
{
    for (int i = 0; i < rockCount; i++)
    {
        Vector3 pos = RandomPointOutsideTrail();
        Quaternion rot = Quaternion.Euler(0f, Random.Range(0f, 360f), 0f);

        GameObject rock = Instantiate(rockPrefab, pos, rot, rocksParent);

        float s = Random.Range(rockScaleRange.x, rockScaleRange.y);
        rock.transform.localScale = new Vector3(s, s * 0.6f, s);
    }
}

void SpawnTrees()
{
    for (int i = 0; i < treeCount; i++)
    {
        Vector3 pos = RandomPointOutsideTrail();
        Quaternion rot = Quaternion.Euler(0f, Random.Range(0f, 360f), 0f);

        GameObject tree = Instantiate(treePrefab, pos, rot, treesParent);

        float s = Random.Range(treeScaleRange.x, treeScaleRange.y);
        tree.transform.localScale = tree.transform.localScale * s;
    }
}

void SpawnFlowersOnTrail()
{
    // Flowers go *on* the trail corridor to guide the player forward.
    for (int i = 0; i < flowerCount; i++)
    {
        float z = Random.Range(-areaHalfSize * 0.8f, areaHalfSize * 0.8f);
        float x = Random.Range(-trailHalfWidth * 0.6f, trailHalfWidth * 0.6f);

        Vector3 pos = new Vector3(x, 0.15f, z);
        Quaternion rot = Quaternion.identity;

        Instantiate(flowerPrefab, pos, rot, flowersParent);
    }
}

Vector3 RandomPointOutsideTrail()
{
    // Rejection sampling: keep generating points until outside the trail corridor.
```

```
        for (int attempts = 0; attempts < 200; attempts++)
        {
            float x = Random.Range(-areaHalfSize, areaHalfSize);
            float z = Random.Range(-areaHalfSize, areaHalfSize);

            // Trail corridor is centered on X=0, extends along Z.
            if (Mathf.Abs(x) > trailHalfWidth)
            {
                return new Vector3(x, 0f, z);
            }
        }

        // Fallback: if something goes wrong, just place at edge
        return new Vector3(trailHalfWidth + 2f, 0f, 0f);
    }
}



// ===== END OF FILE: EnvironmentSpawner.cs =====
```

FollowCamera.cs

Exercise: Exercise 1: Nature Scene

Class: FollowCamera

Purpose: Implements a simple follow camera behaviour that maintains an offset from a target and updates smoothly.

Code:

```
// ===== START OF FILE: FollowCamera.cs =====

using UnityEngine;

public class FollowCamera : MonoBehaviour
{
    public Transform target;
    public Vector3 offset = new Vector3(0f, 4f, -7f);
    public float smoothSpeed = 6f;

    void LateUpdate()
    {
        if (target == null) return;

        Vector3 desiredPos = target.position + offset;
        transform.position = Vector3.Lerp(transform.position, desiredPos, smoothSpeed * Time.deltaTime);
        transform.LookAt(target.position);
    }
}

// ===== END OF FILE: FollowCamera.cs =====
```

PlayerCollector.cs

Exercise: Exercise 1: Nature Scene

Class: PlayerCollector

Purpose: Handles collectable pickup logic and updates gameplay state such as a collected count and any related UI feedback.

Code:

```
// ===== START OF FILE: PlayerCollector.cs =====

using UnityEngine;

public class PlayerCollector : MonoBehaviour
{
    public int collected = 0;

    public void AddCollectable()
    {
        collected++;
        Debug.Log("Collected: " + collected);
    }

    private void OnTriggerEnter(Collider other)
    {
        if (other.CompareTag("Collectable"))
        {
            AddCollectable();
            Destroy(other.gameObject);
        }
    }
}


// ===== END OF FILE: PlayerCollector.cs =====
```

PlayerMoveTransform.cs

Exercise: Exercise 1: Nature Scene

Class: PlayerMoveTransform

Purpose: Implements player movement using transforms, with forward motion and turning from keyboard input and frame rate independent updates.

Code:

```
// ===== START OF FILE: PlayerMoveTransform.cs =====

using UnityEngine;

public class PlayerMoveTransform : MonoBehaviour
{
    public float moveSpeed = 5f;
    public float turnSpeed = 120f;

    void Update()
    {
        float move = Input.GetAxis("Vertical");     // W/S or Up/Down
        float turn = Input.GetAxis("Horizontal");   // A/D or Left/Right

        transform.Rotate(0f, turn * turnSpeed * Time.deltaTime, 0f);

        Vector3 forwardMove = transform.forward * (move * moveSpeed * Time.deltaTime);
        transform.position += forwardMove;
    }
}


// ===== END OF FILE: PlayerMoveTransform.cs =====
```

ThirdPersonCamera.cs

Exercise: Exercise 1: Nature Scene

Class: ThirdPersonCamera

Purpose: Implements a third person camera that follows a target with orbit controls, pitch limits, and smoothing for position and rotation.

Code:

```
// ===== START OF FILE: ThirdPersonCamera.cs =====

using UnityEngine;

public class ThirdPersonCamera : MonoBehaviour
{
    public Transform target;

    [Header("Distance / Height")]
    public float distance = 7f;
    public float height = 4f;

    [Header("Rotation")]
    public float mouseSensitivity = 120f;
    public float minPitch = -10f;
    public float maxPitch = 60f;

    [Header("Smoothing")]
    public float positionSmooth = 8f;
    public float rotationSmooth = 10f;

    float yaw;
    float pitch = 20f;

    void Start()
    {
        if (target == null) return;

        Vector3 angles = transform.eulerAngles;
        yaw = angles.y;
        pitch = Mathf.Clamp(angles.x, minPitch, maxPitch);

        // Optional: lock cursor for nicer control
        Cursor.lockState = CursorLockMode.Locked;
        Cursor.visible = false;
    }

    void LateUpdate()
    {
        if (target == null) return;

        // Mouse input
        float mx = Input.GetAxis("Mouse X");
        float my = Input.GetAxis("Mouse Y");
```

```
        yaw += mx * mouseSensitivity * Time.deltaTime;
        pitch -= my * mouseSensitivity * Time.deltaTime;
        pitch = Mathf.Clamp(pitch, minPitch, maxPitch);

        Quaternion desiredRot = Quaternion.Euler(pitch, yaw, 0f);

        // Desired camera position behind the target
        Vector3 desiredPos = target.position - (desiredRot * Vector3.forward * distance) + Vector3.up * height;

        transform.position = Vector3.Lerp(transform.position, desiredPos, positionSmooth * Time.deltaTime);
        transform.rotation = Quaternion.Slerp(transform.rotation, desiredRot, rotationSmooth * Time.deltaTime);
    }
}


// ===== END OF FILE: ThirdPersonCamera.cs =====
```

Exercise 2: Mini Golf Physics Scene (Rigidbody and Forces)

Scripts extracted from Assets/Scripts in the Mini Golf project.

BallAndAimPutt.cs

Exercise: Exercise 2: Mini Golf

Class: BallAimAndPutt

Purpose: Implements mouse based aiming with raycasting, a chargeable power mechanic, an aim line visualisation, a UI power indicator, and an impulse based shot applied through the Rigidbody.

Code:

```
// ===== START OF FILE: BallAndAimPutt.cs =====

using UnityEngine;
using UnityEngine.UI;

[RequireComponent(typeof(Rigidbody))]
public class BallAimAndPutt : MonoBehaviour
{
    [Header("Power")]
    public float maxPower = 12f;
    public float chargeRate = 10f;

    [Header("Control Rules")]
    public float stopSpeedThreshold = 0.2f;

    [Header("Aiming")]
    public LayerMask aimSurfaceMask;
    public float aimRayDistance = 200f;

    [Header("Visuals")]
    public LineRenderer aimLine;
    public float lineLengthAtMaxPower = 4f;
    public Slider powerSlider;

    Rigidbody rb;
    Camera cam;

    Vector3 aimDir = Vector3.forward;
    float power;
    bool isCharging;

    Vector3 pendingImpulse;
    bool hasPendingImpulse;

    void Awake()
    {
        rb = GetComponent<Rigidbody>();
        cam = Camera.main;
```

```
    if (aimLine != null)
    {
        aimLine.positionCount = 2;
        aimLine.enabled = false;
    }

    if (powerSlider != null)
    {
        powerSlider.minValue = 0f;
        powerSlider.maxValue = maxPower;
        powerSlider.value = 0f;
    }
}

void Update()
{
    // Do not allow aiming while the ball is moving
    if (rb.linearVelocity.magnitude > stopSpeedThreshold)
    {
        ResetAimUI();
        return;
    }

    UpdateAimDirectionFromMouse();

    if (Input.GetMouseButtonDown(0))
    {
        isCharging = true;
        power = 0f;
    }

    if (isCharging && Input.GetMouseButton(0))
    {
        power = Mathf.Clamp(power + chargeRate * Time.deltaTime, 0f, maxPower);
    }

    if (isCharging && Input.GetMouseButtonUp(0))
    {
        QueueShot();
        isCharging = false;
        power = 0f;
    }

    UpdateAimUI();
}

void FixedUpdate()
{
    if (!hasPendingImpulse) return;

    rb.AddForce(pendingImpulse, ForceMode.Impulse);
    hasPendingImpulse = false;
}

void UpdateAimDirectionFromMouse()
{
    if (cam == null) return;
```

```
        Ray ray = cam.ScreenPointToRay(Input.mousePosition);
        if (Physics.Raycast(ray, out RaycastHit hit, aimRayDistance, aimSurfaceMask))
        {
            Vector3 flat = hit.point - transform.position;
            flat.y = 0f;

            if (flat.sqrMagnitude > 0.001f)
            {
                aimDir = flat.normalized;
            }
        }
    }

    void QueueShot()
    {
        pendingImpulse = aimDir * power;
        hasPendingImpulse = true;
    }

    void UpdateAimUI()
    {
        if (aimLine != null)
        {
            aimLine.enabled = true;

            Vector3 start = transform.position + Vector3.up * 0.2f;
            float t = maxPower > 0f ? (power / maxPower) : 0f;
            Vector3 end = start + aimDir * (lineLengthAtMaxPower * t);

            aimLine.SetPosition(0, start);
            aimLine.SetPosition(1, end);
        }

        if (powerSlider != null)
        {
            powerSlider.value = power;
        }
    }

    void ResetAimUI()
    {
        isCharging = false;
        power = 0f;

        if (aimLine != null) aimLine.enabled = false;
        if (powerSlider != null) powerSlider.value = 0f;
    }
}


// ===== END OF FILE: BallAndAimPutt.cs =====
```

HoleWin.cs

Exercise: Exercise 2: Mini Golf

Class: HoleWin

Purpose: Detects when the ball enters the hole trigger and triggers win feedback such as on screen text and optional ball stopping.

Code:

```
// ===== START OF FILE: HoleWin.cs =====

using UnityEngine;

public class HoleWin : MonoBehaviour
{
    public GameObject winText; // optional UI
    public Rigidbody ballRb;

    void Start()
    {
        if (winText != null) winText.SetActive(false);
    }

    void OnTriggerEnter(Collider other)
    {
        if (!other.CompareTag("Ball")) return;

        if (winText != null) winText.SetActive(true);

        // Optional: stop ball when in hole
        if (ballRb != null)
        {
            ballRb.linearVelocity = Vector3.zero;
            ballRb.angularVelocity = Vector3.zero;
        }

        Debug.Log("Ball in the hole!");
    }
}


// ===== END OF FILE: HoleWin.cs =====
```

SimpleFollowCamera.cs

Exercise: Exercise 2: Mini Golf

Class: SimpleFollowCamera

Purpose: Follows a target from a fixed offset and smooths camera motion, suitable for a top down mini golf view.

Code:

```
// ===== START OF FILE: SimpleFollowCamera.cs =====

using UnityEngine;

public class SimpleFollowCamera : MonoBehaviour
{
    public Transform target;
    public Vector3 offset = new Vector3(0f, 18f, -18f);
    public float smooth = 5f;

    void LateUpdate()
    {
        if (target == null) return;

        Vector3 desired = target.position + offset;
        transform.position = Vector3.Lerp(transform.position, desired, smooth * Time.deltaTime);
        transform.LookAt(target.position);
    }
}


// ===== END OF FILE: SimpleFollowCamera.cs =====
```